嵌入式系統 - Lab 9

Install opencv and Deep learning

---

1. 實驗目的

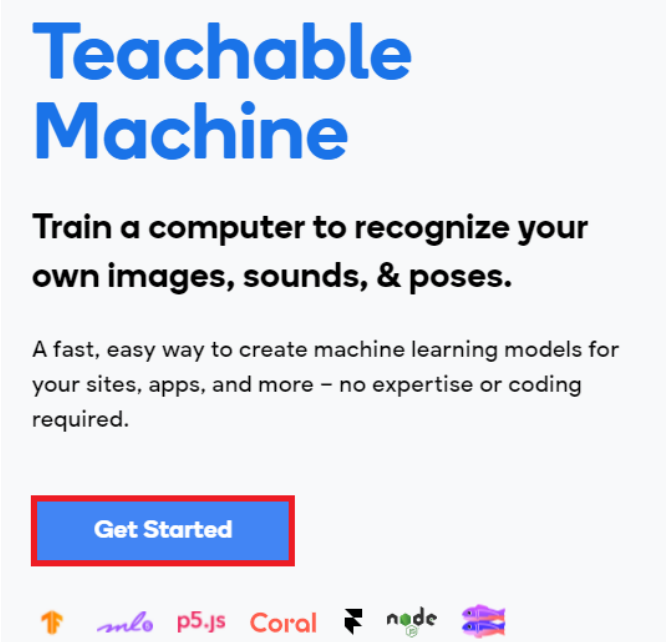
在 Raspberry Pi 4 上安裝 opencv 和 deep learning，並訓練自己的資料集去偵測物件。

2. 實驗環境

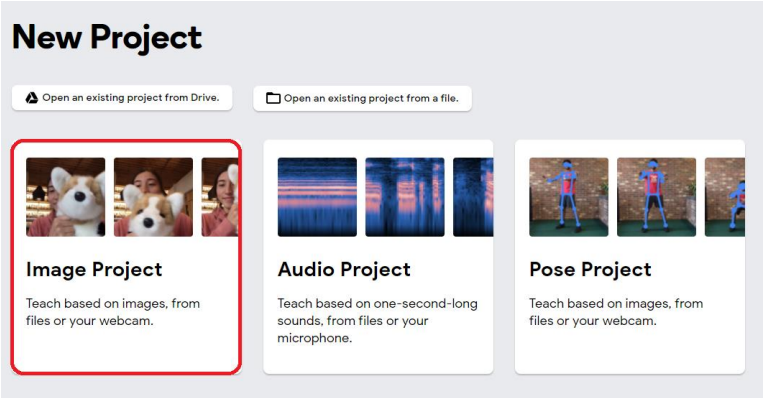
PC 端 OS：Windows 10
Raspberry Pi 4 OS：5.10.39-71+

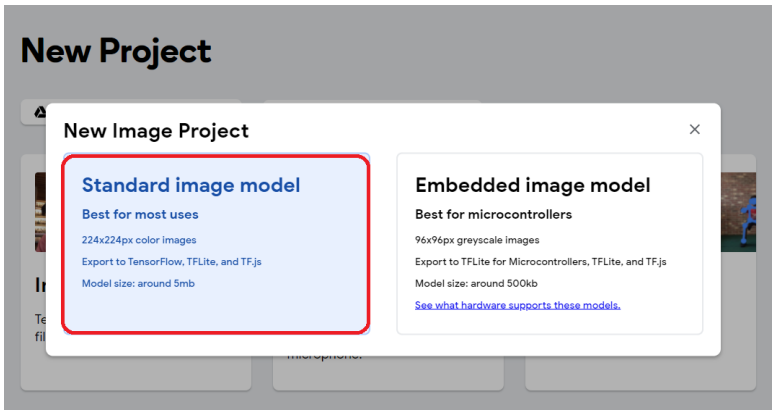
3. 操作步驟

[1]. 使用 Teachable Machine 來進行資料的訓練

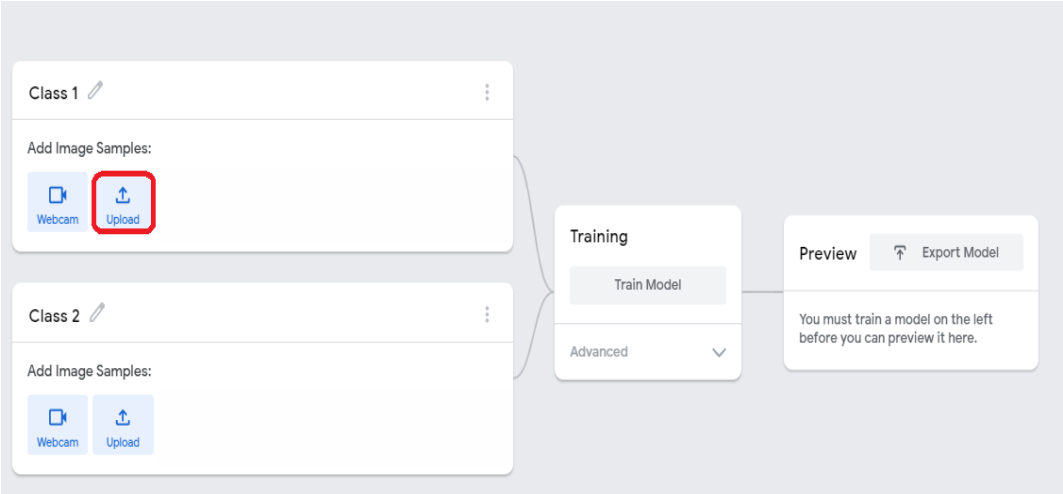


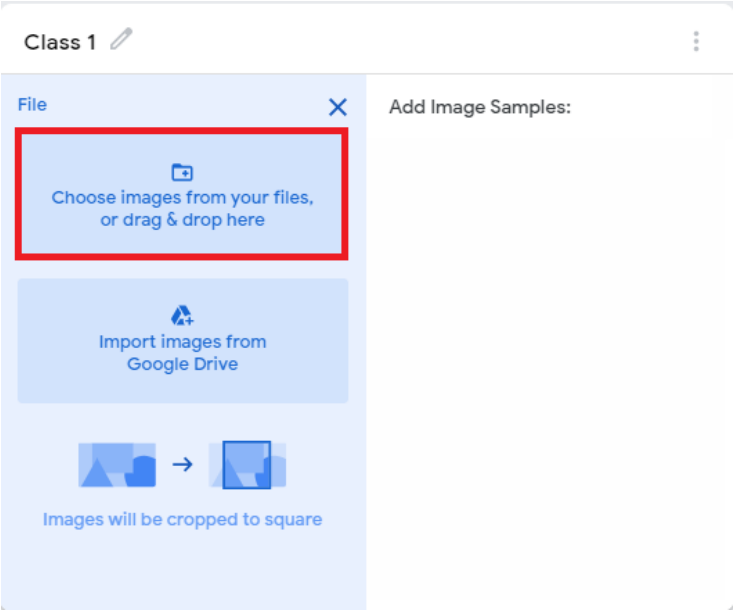
- 點選 Get Started 後選擇 Image Project，並選擇 Standard Image model。



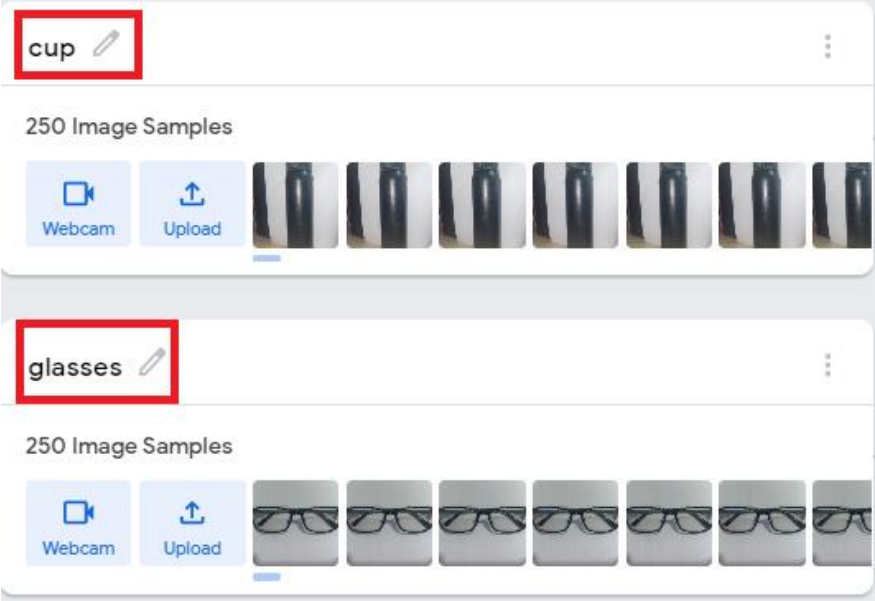


- 訓練資料可以使用 webcam 拍攝或選擇 upload 上傳，本文使用 upload。這裡可以使用 Kaggle 的網站獲取自己喜歡的 dataset Kaggle 網址:https://www.kaggle.com/

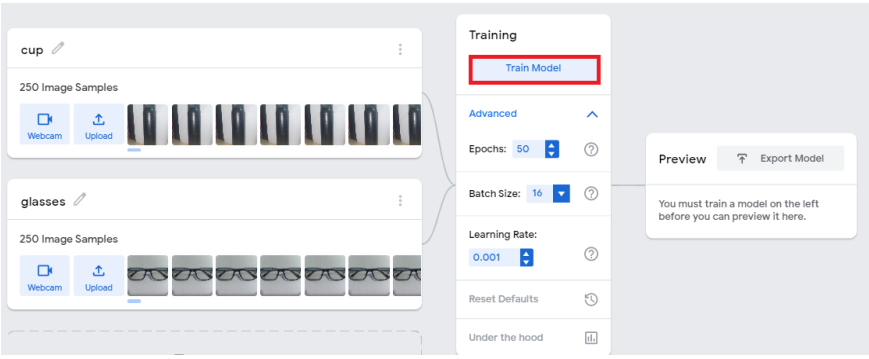




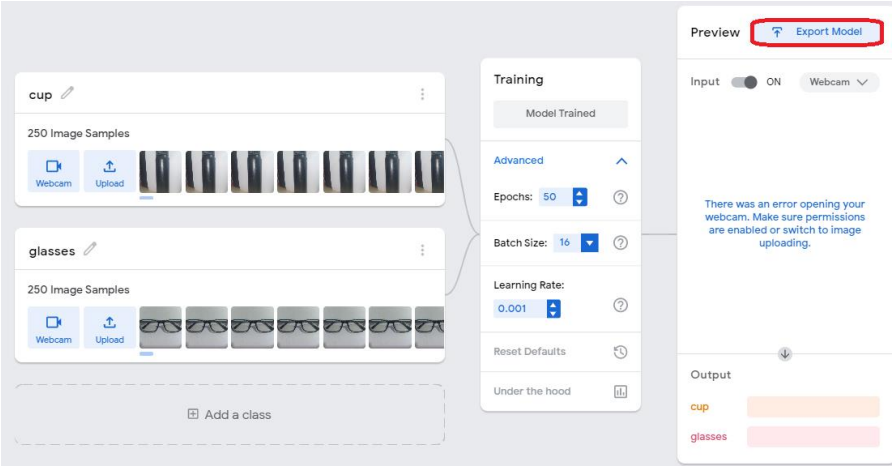
- 照片上傳完畢後更改 Class 名稱以便後續辨識時做顯示。
- 建議上傳一個乾淨的照片作為 nothing 的訓練，此 class 為沒有任何物件的圖片。



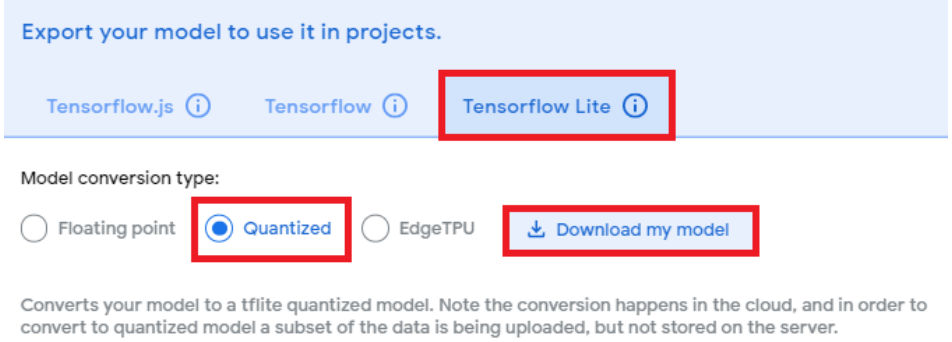
- 點選 Training 做資料訓練，設定皆為預設。



- 訓練完畢後選擇 Export Model。

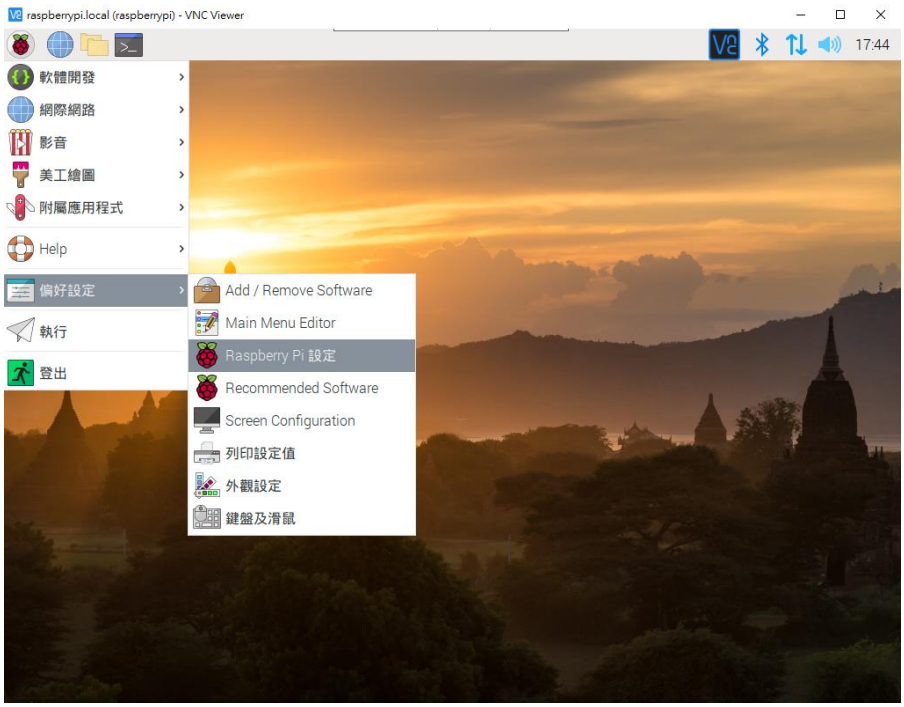


- 選擇 Tensorflow Lite 後選擇 Quantized，最後 Download my model。

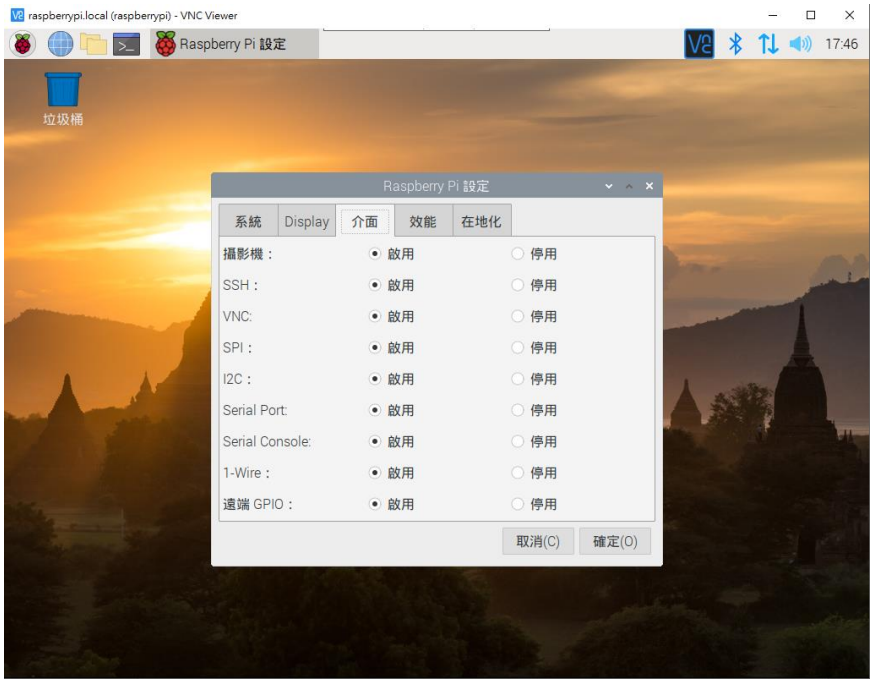


[2]. 安裝及設定 OpenCV

- 使用 VNC 進入 Raspberrypi,點選 Raspberry Pi 圖示,選擇偏好設定,再點選 Raspberry Pi 設定

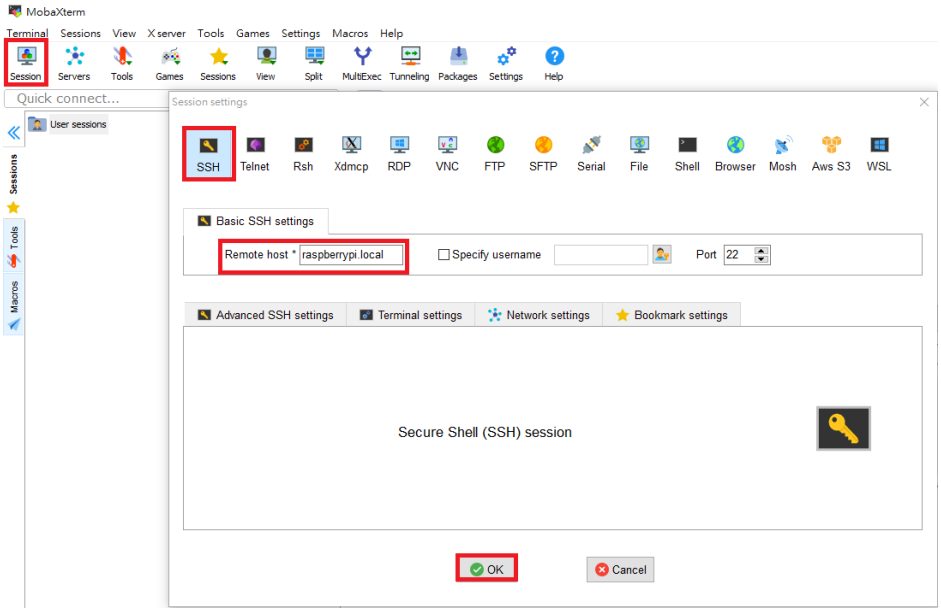


- 選擇介面，將所有選項啟用並重新開機。

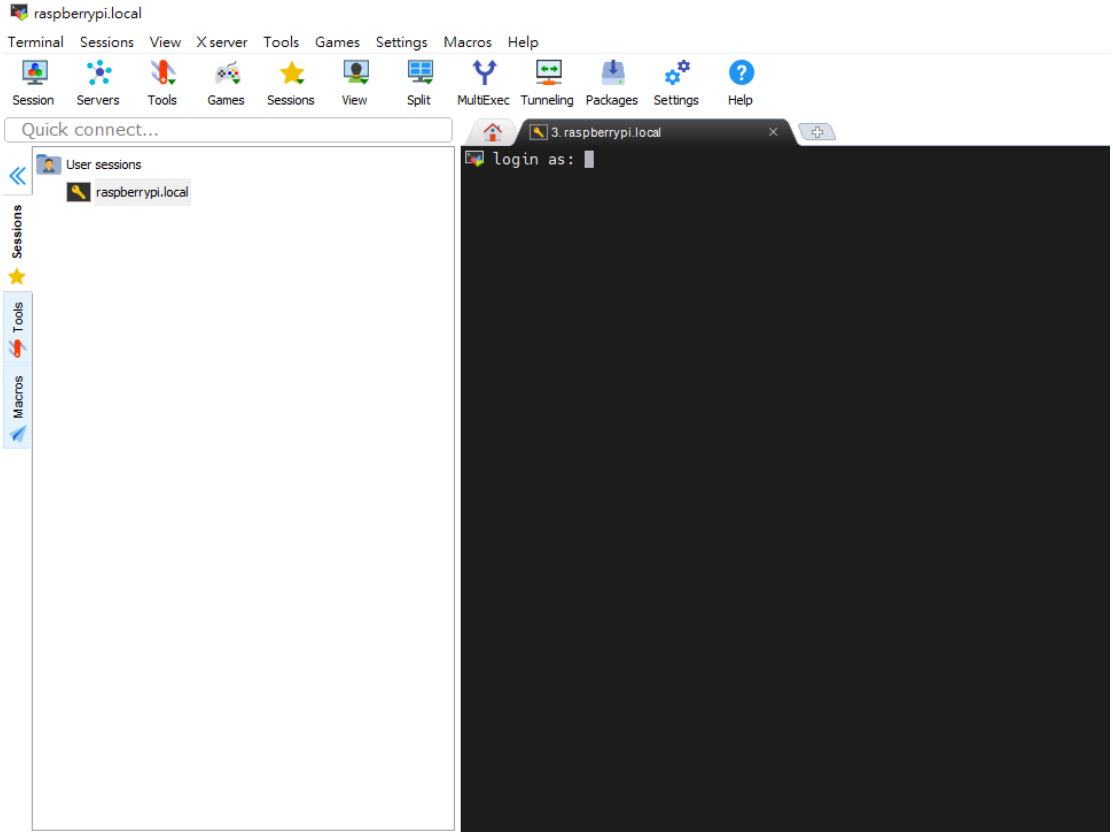


[3]. 安裝 MobaXterm

- 連結：https://mobaxterm.mobatek.net/download.html
- 此軟體為後續放入訓練資料所需。(安裝於 win10)
- 選擇 Seesion 後選擇 SSH 並於 Remote host 內輸入 mDNS 進行控制。



- 輸入預設帳號密碼。



[4]. 安裝 Tensorflow Lite

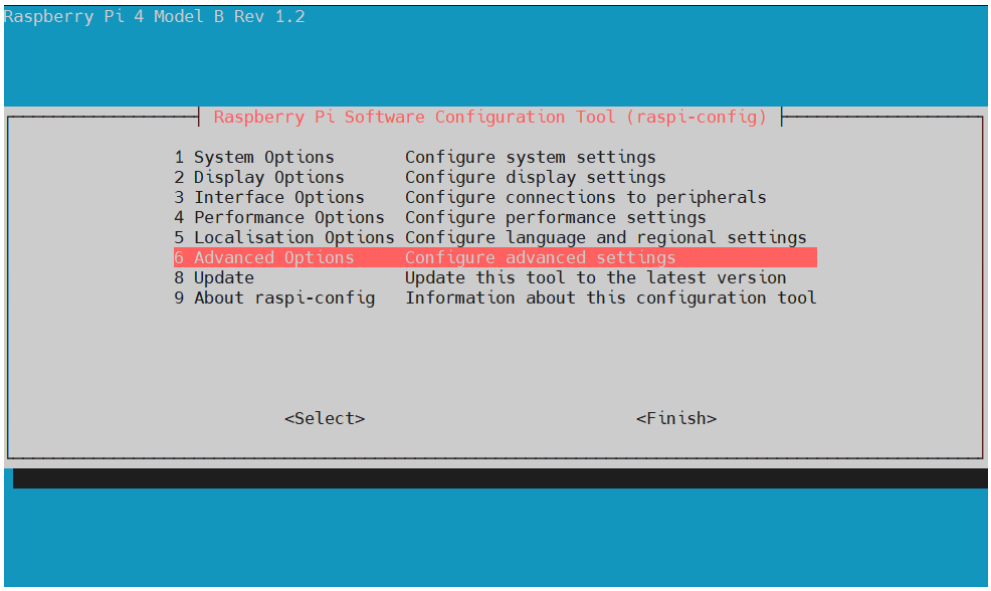
pip3 install https://dl.google.com/coral/python/tflite_runtime-2.1.0.post1-cp37-cp37m-linux_armv7l.whl

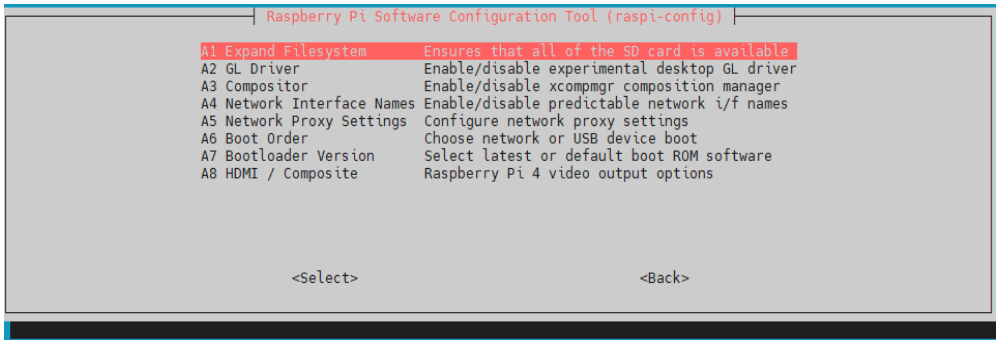
```
pi@raspberrypi:~ $ pip3 install https://dl.google.com/coral/python/tflite_runtime-2.1.0.post1-cp37-cp37m-linux_armv7l.whl
```

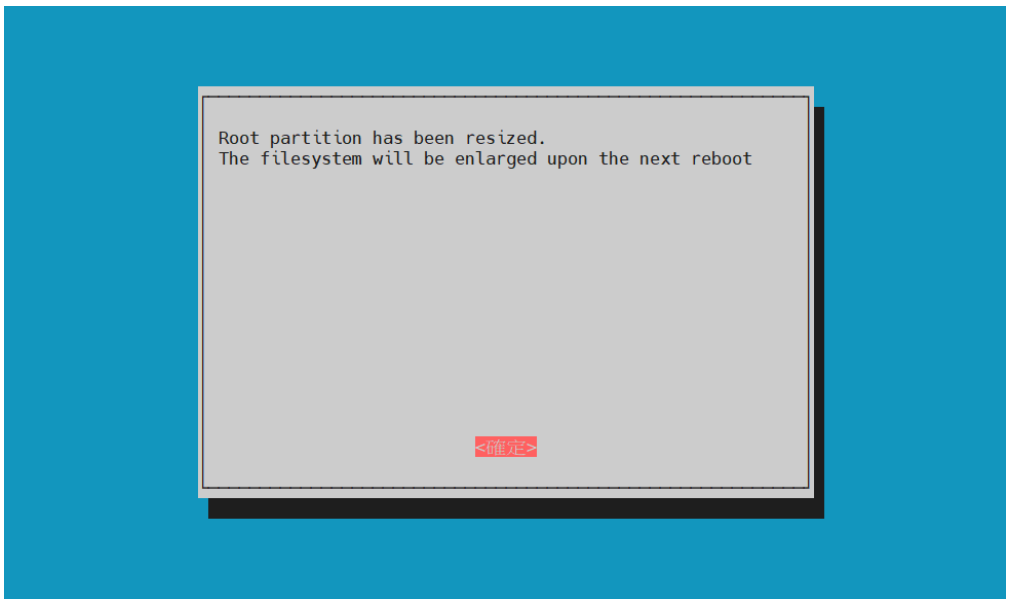
- 擴展文件系統到整張 SD 卡，輸入

```
sudo raspi-config
```

- 選擇 Advanced Options 後選擇 Expand Filesystem，選擇完畢後重新開機。







- 更新並升級所有套件包

sudo apt-get update && sudo apt-get upgrade

```
pi@raspberrypi:~ $ sudo apt-get update && sudo apt-get upgrade
```

- 安裝 Cmake 套件

sudo apt-get install libjpeg-dev libpng-dev libtiff-dev libjasper-dev libwebp-dev libopenexr-dev
sudo apt-get install libavcodec-dev libavformat-dev libswscale-dev libv4l-dev libxvidcore-dev libx264-dev libdc1394-22-dev libgstreamer-plugins-base1.0-dev libgstreamer1.0-dev
sudo apt-get install libgtk-3-dev libqtgui4 libqtwebkit4 libqt4-test python3-pyqt5
sudo apt-get install libatlas-base-dev liblapacke-dev gfortran libcanberra-gtk*
sudo apt-get install libhdf5-dev libhdf5-103
sudo apt-get install python3-dev python3-pip python3-numpy

```
pi@raspberrypi:~ $ sudo apt-get install libjpeg-dev libpng-dev libtiff-dev libjasper-dev libwebp-d
ev libopenexr-dev
```

```
pi@raspberrypi:~ $ sudo apt-get install libavcodec-dev libavformat-dev libswscale-dev libv4l-dev l
ibxvidcore-dev libx264-dev libdc1394-22-dev libgstreamer-plugins-base1.0-dev libgstreamer1.0-dev
```

```
pi@raspberrypi:~ $ sudo apt-get install libgtk-3-dev libqtgui4 libqtwebkit4 libqt4-test python3-py
qt5
```

```
pi@raspberrypi:~ $ sudo apt-get install libatlas-base-dev liblapacke-dev gfortran libcanberra-gtk*
```

```
pi@raspberrypi:~ $ sudo apt-get install libhdf5-dev libhdf5-103
```

```
pi@raspberrypi:~ $ sudo apt-get install python3-dev python3-pip python3-numpy
```

[5]. 下載 OpenCV4.0

wget -O opencv.zip https://github.com/opencv/opencv/archive/4.0.0.zip
wget -O opencv_contrib.zip https://github.com/opencv/opencv_contrib/archive/4.0.0.zip

```
pi@raspberrypi:~ $ wget -O opencv.zip https://github.com/opencv/opencv/archive/4.0.0.zip
```

```
pi@raspberrypi:~ $ wget -O opencv_contrib.zip https://github.com/opencv/opencv_contrib/aive/4.0.0.
zip
```

- 解壓縮檔案

unzip opencv.zip
unzip opencv_contrib.zip

```
pi@raspberrypi:~ $ unzip opencv.zip
```

```
pi@raspberrypi:~ $ unzip opencv_contrib.zip
```

- 建立 opencv 和 opencv_contrib 資料夾及將檔案放置資料夾內

mv opencv-4.0.0 opencv
mv opencv_contrib-4.0.0 opencv_contrib

```
pi@raspberrypi:~ $ mv opencv-4.0.0 opencv
```

```
pi@raspberrypi:~ $ mv opencv_contrib-4.0.0 opencv_contrib
```

- 在 opencv 資料夾內建立名為 build 的資料夾

cd ~/opencv
mkdir build
cd build

```
pi@raspberrypi:~ $ cd ~/opencv
pi@raspberrypi:~/opencv $ mkdir build
pi@raspberrypi:~/opencv $ cd build
pi@raspberrypi:~/opencv/build $
```

- 安裝 Cmake 套件

sudo apt-get install build-essential cmake

- 使用 CMake 來設置 OpenCV 4 環境

cmake -D CMAKE_BUILD_TYPE=RELEASE \
-D CMAKE_INSTALL_PREFIX=/usr/local \
-D OPENCV_EXTRA_MODULES_PATH=~/opencv_contrib/modules \
-D ENABLE_NEON=ON \
-D ENABLE_VFPV3=ON \
-D BUILD_TESTS=OFF \
-D OPENCV_ENABLE_NONFREE=ON \
-D INSTALL_PYTHON_EXAMPLES=OFF \
-D CMAKE_SHARED_LINKER_FLAGS=-latomic \
-D BUILD_EXAMPLES=OFF ..

注意最後一行指令後面有個 ..

```
pi@raspberrypi:~/opencv/build $ cmake -D CMAKE_BUILD_TYPE=RELEASE \
> -D CMAKE_INSTALL_PREFIX=/usr/local \
> -D OPENCV_EXTRA_MODULES_PATH=~/opencv_contrib/modules \
> -D ENABLE_NEON=ON \
> -D ENABLE_VFPV3=ON \
> -D BUILD_TESTS=OFF \
> -D OPENCV_ENABLE_NONFREE=ON \
> -D INSTALL_PYTHON_EXAMPLES=OFF \
> -D BUILD_EXAMPLES=OFF ..
```

- 調整 RPi4 的 SWAP 交換空間，來解決編譯 OpenCV 記憶體不足的問題 CONF_SWAPSIZE=100 改成 2048

sudo nano /etc/dphys-swapfile

```
# /etc/dphys-swapfile - user settings for dphys-swapfile package
# author Neil Franklin, last modification 2010.05.05
# copyright ETH Zuerich Physics Departement
#   use under either modified/non-advertising BSD or GPL license

# this file is sourced with . so full normal sh syntax applies

# the default settings are added as commented out CONF_*=* lines


# where we want the swapfile to be, this is the default
#CONF_SWAPFILE=/var/swap

# set size to absolute value, leaving empty (default) then uses computed value
#   you most likely don't want this, unless you have an special disk situation
CONF_SWAPSIZE=100

# set size to computed value, this times RAM size, dynamically adapts,
#   guarantees that there is enough swap without wasting disk space on excess
#CONF_SWAPFACTOR=2

# restrict size (computed and absolute!) to maximally this limit
#   can be set to empty for no limit, but beware of filled partitions!
#   this is/was a (outdated?) 32bit kernel limit (in MBytes), do not overrun it
#   but is also sensible on 64bit to prevent filling /var or even / partition
#CONF_MAXSWAP=2048
```

- 重新開啟 SWAP 服務

sudo /etc/init.d/dphys-swapfile stop
sudo /etc/init.d/dphys-swapfile start

```
pi@raspberrypi:~/opencv/build $ sudo /etc/init.d/dphys-swapfile stop
```

```
pi@raspberrypi:~/opencv/build $ sudo /etc/init.d/dphys-swapfile start
```

- 開啟四核心編譯 OpenCV(此步驟需要耗時，請耐心等待)

make -j4

```
pi@raspberrypi:~/opencv/build $ make -j4
```

- 安裝 OpenCV

sudo make install
sudo ldconfig

```
pi@raspberrypi:~/opencv/build $ sudo make install
```

```
pi@raspberrypi:~/opencv/build $ sudo ldconfig
```

- CONF_SWAPSIZE 2048 改成 100

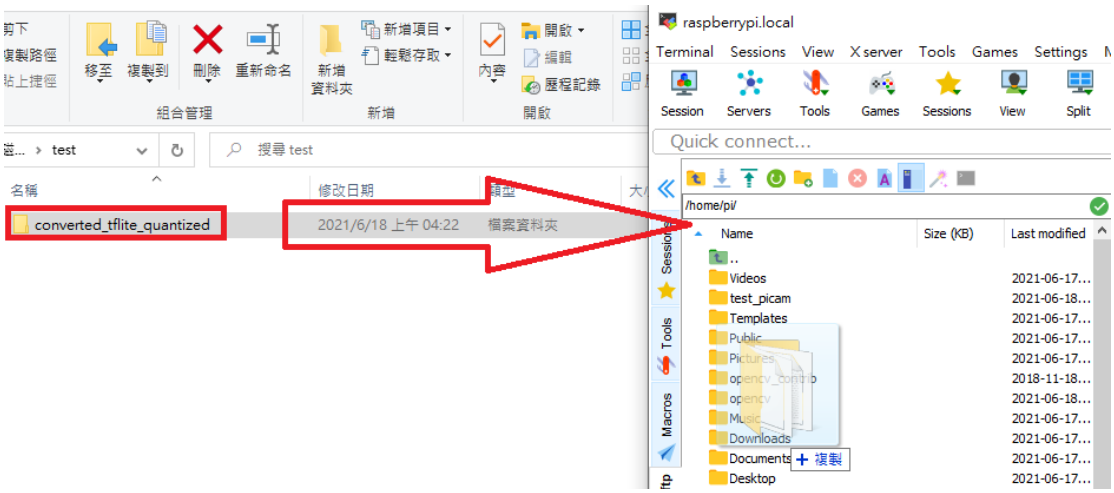
sudo nano /etc/dphys-swapfile

- 重新開啟 SWAP 服務

sudo /etc/init.d/dphys-swapfile stop
sudo /etc/init.d/dphys-swapfile start

[6]. 偵測

- 將訓練出來的模型放入 Raspberry Pi



- 匯入 OpenCV 檔於專案資料夾內

cd ~/converted_tflite_quantized/
ln –s /usr/local/python/cv2/python-3.7/cv2.cpython-37m-arm-linux-gnueabihf.so cv2.so

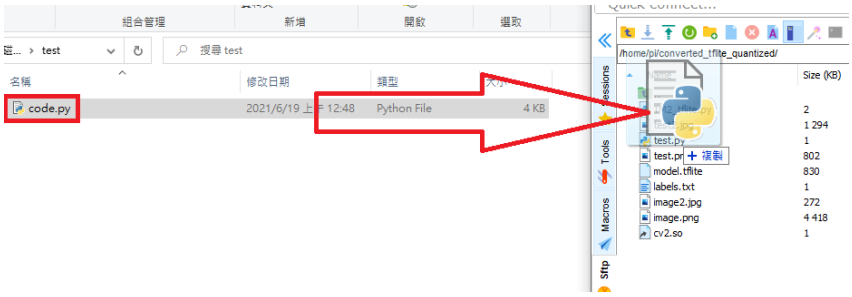
```
pi@raspberrypi:~/opencv/build $ cd ~/converted_tflite_quantized/
pi@raspberrypi:~/converted_tflite_quantized $ ln -s /usr/local/python/cv2/python-3.7/cv2.cpython-3
7m-arm-linux-gnueabihf.so cv2.so
```

- 測試 OpenCV

python3
import cv2
cv2.__version__
exit()

```
pi@raspberrypi:~/converted_tflite_quantized $ python3
Python 3.7.3 (default, Jan 22 2021, 20:04:44)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cv2
cv>>> cv2.__version__
'4.0.0'
>>> exit()
```

- 將程式碼放置於訓練資料夾內



- 連接上 Webcam 至 Raspberry Pi USB 中，並安裝 fswebcam

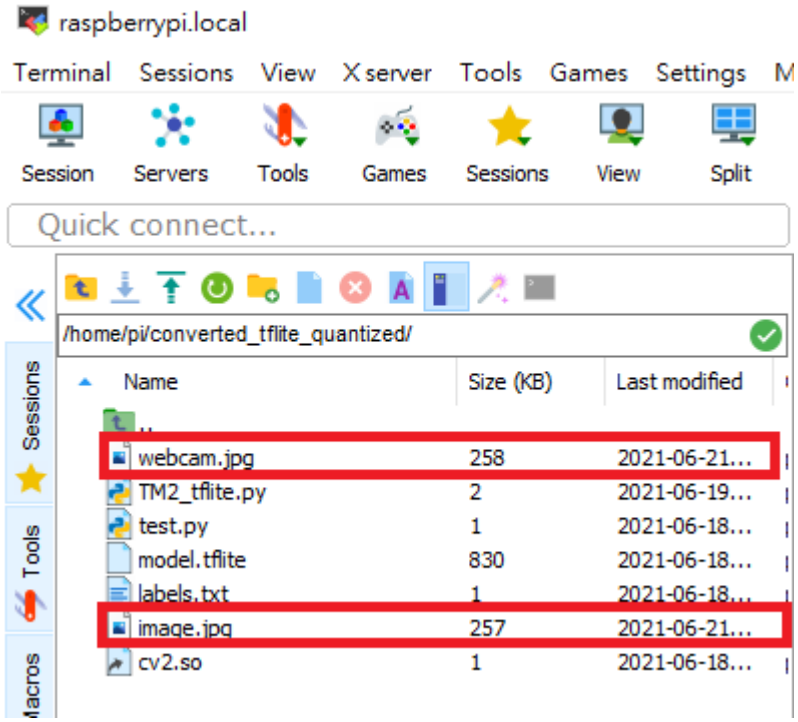
sudo apt-get install fswebcam

```
pi@raspberrypi:~/converted_tflite_quantized $ sudo apt-get install fswebcam
```

- 測試是否有連接成功，上面的指令是直接拍照，下面的指令是可以指定 /dev/video0 這台網路攝影機，跳過 10 個 frames，照一張 640x360 大小的照片，用於多個硬體時使用。

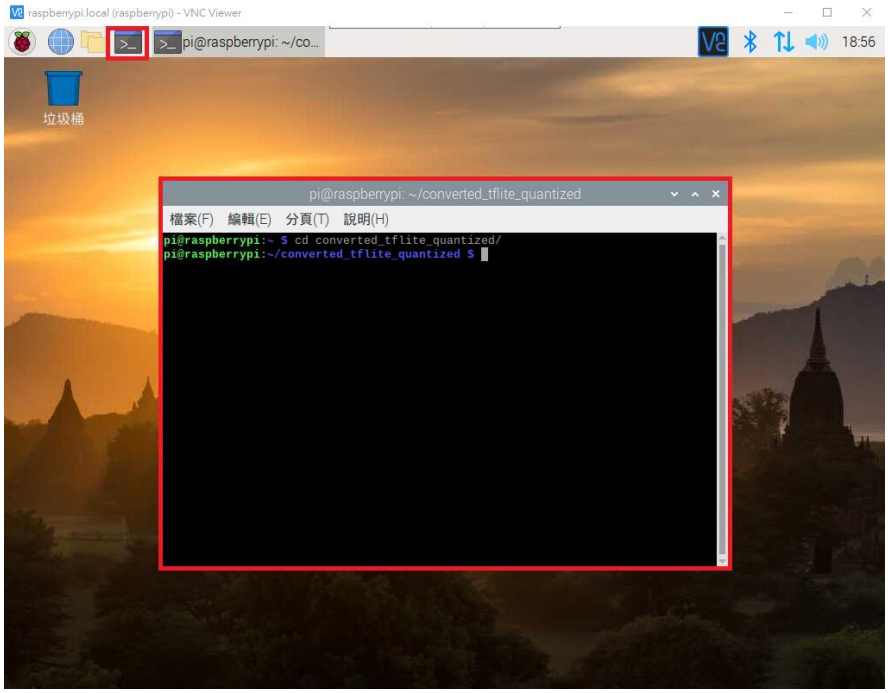
fswebcam image.jpg
fswebcam -r 640x360 -S 10 -d /dev/video0 webcam.jpg

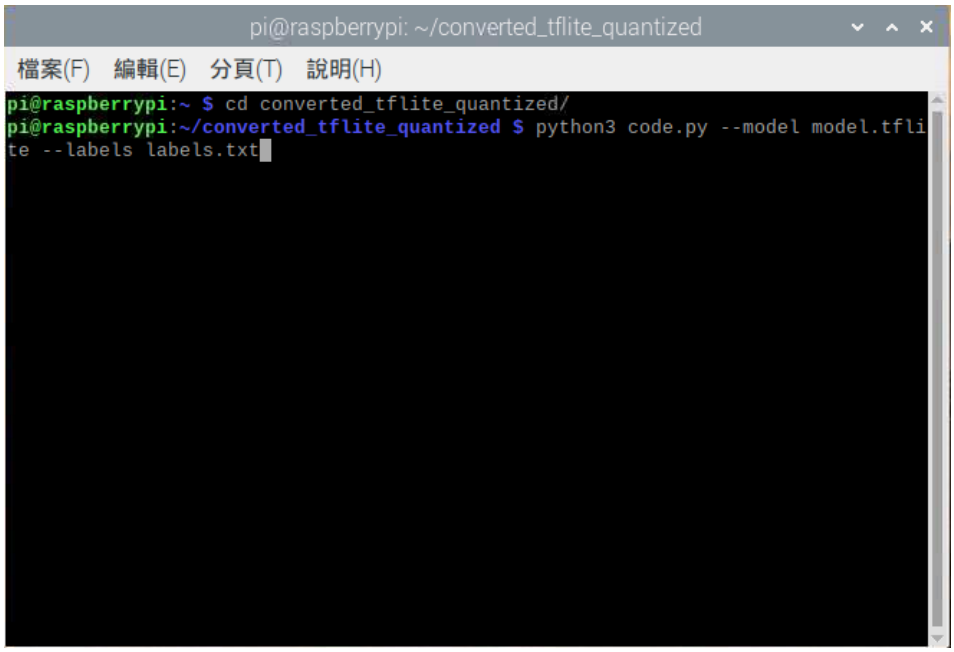
```
pi@raspberrypi:~/converted_tflite_quantized $ fswebcam image.jpg
--- Opening /dev/video0...
Trying source module v4l2...
/dev/video0 opened.
No input was specified, using the first.
Adjusting resolution from 384x288 to 640x480.
--- Capturing frame...
Captured frame in 0.00 seconds.
--- Processing captured image...
Writing JPEG image to 'image.jpg'.
pi@raspberrypi:~/converted_tflite_quantized $ fswebcam -r 640x360 -S 10 -d /dev/video0 webcam.jpg
--- Opening /dev/video0...
Trying source module v4l2...
/dev/video0 opened.
No input was specified, using the first.
Adjusting resolution from 640x360 to 640x480.
--- Capturing frame...
Skipping 10 frames...
Capturing 1 frames...
Captured 11 frames in 0.90 seconds. (12 fps)
--- Processing captured image...
Writing JPEG image to 'webcam.jpg'.
```

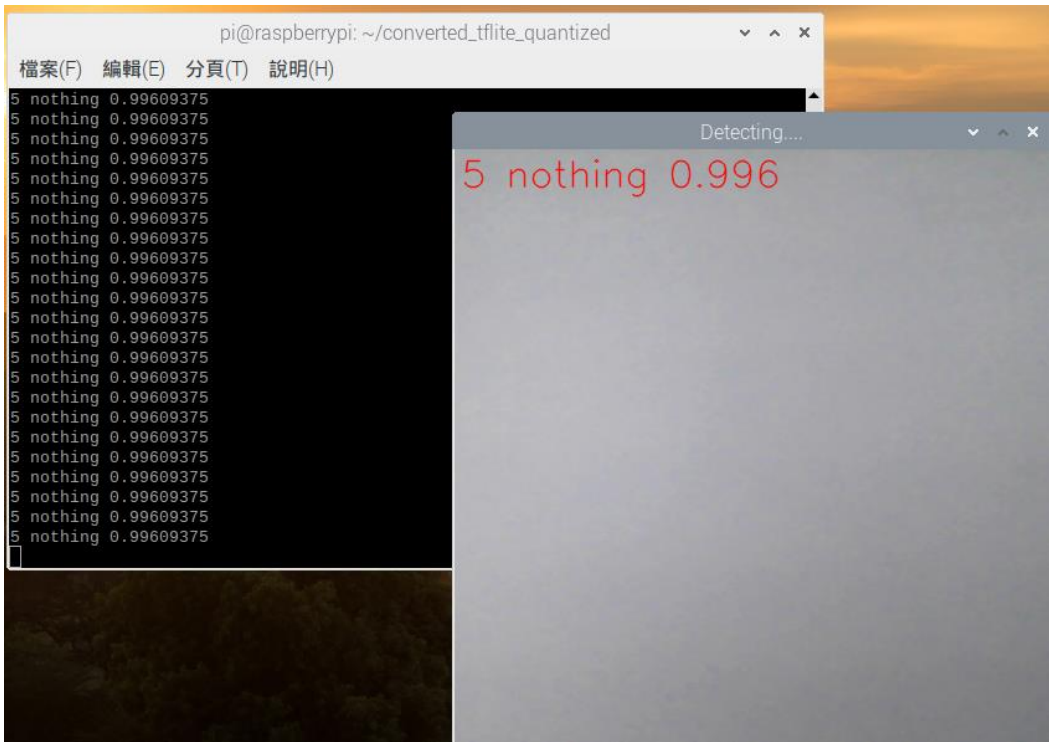


- 使用 VNC Viewer 連入 Raspberry Pi，開啟 Terminal 輸入指令即可成功執行。

cd ~/converted_tflite_quantized/
python3 code.py --model model.tflite --labels labels.txt

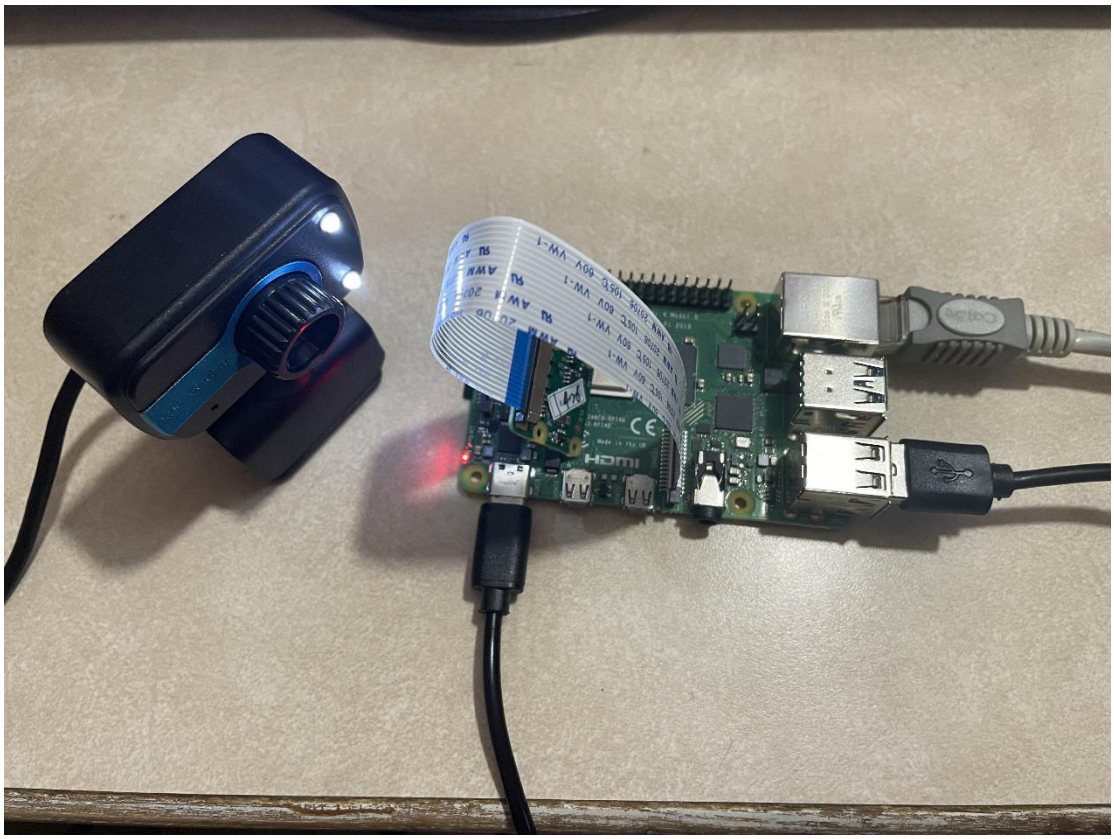


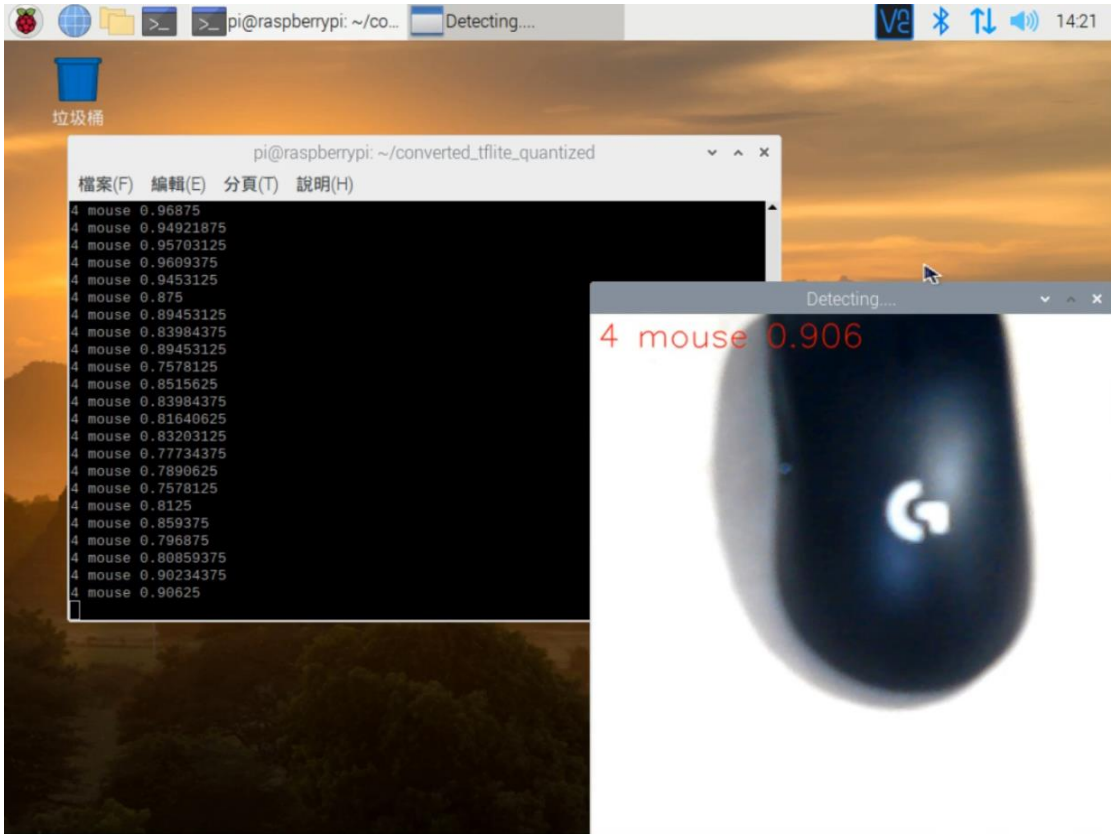

pi@raspberrypi: ~/converted_tflite_quantized
檔案(F) 編輯(E) 分頁(T) 說明(H)
pi@raspberrypi:~ $ cd converted_tflite_quantized/
pi@raspberrypi:~/converted_tflite_quantized $ python3 code.py --model model.tfli
te --labels labels.txt


pi@raspberrypi: ~/converted_tflite_quantized
檔案(F) 編輯(E) 分頁(T) 說明(H)
5 nothing 0.99609375
Detecting....
5 nothing 0.996

-

[7]. Demo





- 網路攝影機照射到的物體會顯示於畫面中，左邊的終端機會持續更新照到的物體及準確率。

- 影片連結:
- https://youtu.be/HFq6MKXG1vI
- https://youtu.be/Lr9JjF-cdgk

4. References

[1] 使用 Google Teachable Machine 來實現 Raspberry Pi 4 的影像分類
https://www.rs-online.com/designspark/google-teachable-machine-raspberry-pi-4-cn

[2] Raspberry Pi 相機模組安裝
https://blog.wuct.me/raspberry-pi-100abbe7a1fd

[3] Object Detection Raspberry Pi using OpenCV Python | 2020
https://www.youtube.com/watch?v=Vg9rrOFmwHo&ab_channel=Murtaza%27sWorkshop-RoboticsandAI

[4] Raspberry Pi 學習筆記(五) : Pi Camera + Python +OpenCV 安裝
https://yanwei-liu.medium.com/raspberry-pi%E5%AD%B8%E7%BF%92%E7%AD%86%E8%A8%98-%E4%BA%94-pi-camera-python-opencv-6613c4ebe8a1

[5] Installing OpenCV on Raspberry Pi 4B
https://www.youtube.com/watch?v=OugQIz_vcFo&ab_channel=cytrontech

[6] 在 Raspberry Pi 中使用 USB 網路攝影機(webcam)照相
https://blog.gtwang.org/iot/raspberry-pi-usb-webcam/